Python Project # 2 - Greedy Algorithm for a Scheduling Problem

In this project we want to use the Greedy algorithm to solve a scheduling problem. Assume that you have a summer internship where you work 8 hours per day. Your supervisor has given you a list of 7 tasks to complete and has prioritized these by ranking them by 1 through 10, where 10 is the most important task. You look at this list of tasks and estimate the time (in minutes) that you need to complete each one. Unfortunately you can't complete all 7 tasks in 8 hours so the goal is to determine which tasks to do.

We first assume that we use a Greedy Algorithm where we choose the task which has the highest priority to complete first. When that is done we choose the task to do which has the next highest priority that can be completed in the remaining time, etc. Your goal is to determine which tasks you can complete in the first 8 hour day.

What is our strategy for writing this code to determine which tasks to do on the first day?

Basically, this is just like the Greedy algorithm for the Knapsack problem where instead of picking the item with the highest value not already in the Knapsack (and which fits), we pick the task which has the highest priority and which is not already completed and which can be done in the remaining time.

We will construct our program in a step-by-step mammner.

Step 1. Input the problem specific data.

- I have defined the two task lists; one is for the priority of each task and the other is for the time in minutes that each task takes.
- Next define the number of tasks. Do not use a number (like 7) but rather use the **len()** command.
- Next define the amount of time in minutes that is available to complete the tasks on the first day.
- Output the information in a table-like format; later we will learn how to do this to make it look nice but for now just print out the task numbers from 1 to 7, their priority and the time to complete the task. I have added the commands to print the title and one line of the table. To print all the tasks you need to put this statement inside a **for** loop and modify it appropriately.

```
priority = [2, 6, 1, 7, 10, 8, 5]
time = [30, 200, 20, 70, 120, 60, 150]
# define number of tasks (don't handwire, use len)
n_tasks = len(priority)
max_time = 8*60
print("TASK      PRIORITY        TIME TO COMPLETE")
for i in range(n_tasks):
    print(f"{i+1}          {priority[i]}            {time[i]}")
```

```
TASK      PRIORITY        TIME TO COMPLETE
1          2           30
2          6           200
3          1           20
4          7           70
5          10           120
6          8           60
7          5           150
```

```
```

```
test_list = [ 4, 5, 3, 7, 11, 2]
max_value = max(test_list)  # use function max() to find maximum value in list
max_index = test_list.index(max_value)  # use method list.index () to find the index where this max occurs
# print out results in formatted statement
print(f"Maximum value = {max_value}, index where it occurs = {max_index}")
```

```
Maximum value = 11, index where it occurs = 4
```

```
test_list = [4,  5, 3, 7, 11, 2]
time_list = [30, 45, 20, 120, 90, 70]
max_value = max(test_list)
max_index = test_list.index(max_value)
task_time = time_list[max_index]
print(f"Task chosen: value {max_value}, time taken to complete: {task_time}")   # print out values as in Step #2 plus the
```

```
Task chosen: value 11, time taken to complete: 90
```

```
max_time = 6 * 60  # this is not the maximum time available for your example but rather for testing
time_used = 3 * 60
#
# Following 6 statements are from Step 3
test_list = [ 4, 5, 3, 7, 11, 2]
time_list = [30, 45, 20, 120, 90, 70]
max_value = max(test_list)
max_index = test_list.index(max_value)
task_time = time_list[max_index]
print (f"task chosen: value {max_value}, time taken to complete: {task_time}")
#
# Add conditional to see if this task can be completed in the allotted time
if (task_time + time_used <= max_time):
    time_used += task_time
    print(f"Added task {max_index+1}, time used = {time_used}")
    # if conditional is true, i.e., task can be completed update time_used
    # add print statements
```

```
task chosen: value 11, time taken to complete: 90
Added task 5, time used = 270
```

```
# Input problem specific data and print it out from Step 1
priority = [ 2, 6, 1, 7, 10, 8, 5]
time = [30,200,20,70,120,60, 150]
n_tasks = len(priority)
max_time = 8*60
print ( f"TASK      PRIORITY      TIME TO COMPLETE")

# Initialize any variables before loop

time_used = 0

# add for loop to go through all tasks
for i in range(n_tasks):
    max_value = max(priority)
    max_index = priority.index(max_value)
    task_time = time[max_index]
# zero out this entry in the priority list so you won't find it again
    priority[max_index] = 0
#add conditional to check to see if task can be completed as in Step 4
    if time_used + task_time <= max_time:
        time_used += task_time
        print(f"{max_index+1}         {max_value}          {time_used}")

# Print out the total time used for all tasks in day 1
print(f"total time used = {time_used}")
```

```
TASK      PRIORITY      TIME TO COMPLETE
5         10          120
6         8          180
4         7          250
2         6          450
1         2          480
total time used = 480
```

```
# Input problem specific data and print it out from Step 1
priority = [ 2, 6, 1, 7, 10, 8, 5]
time = [30,200,20,70,120,60, 150]
n_tasks = len(priority)
max_time = 8*60
print ( f"TASK      PRIORITY      TIME TO COMPLETE")

# Initialize any variables before loop

time_used = 0

# add for loop to go through all tasks

for i in range (n_tasks):
    max_value = max(priority)
    max_index = priority.index(max_value)
    task_time = time[max_index]
    priority[max_index]= 0

    if time_used + task_time <= max_time:
        time_used += task_time
        print(f"{max_index+1}         {max_value}              {time_used}")
        if time_used == max_time:
            break
print(f"total time used = {time_used}")
```

```
TASK      PRIORITY      TIME TO COMPLETE
5         10             120
6         8             180
4         7             250
2         6             450
1         2             480
total time used = 480
```

```
# Input problem specific data
old_priority = [2, 6, 1, 7, 10, 8, 5]
time = [30, 200, 20, 70, 120, 60, 150]
max_time= 8*60
n_tasks = len(old_priority)
print(f"TASK          PRIORITY        TIME TO COMPLETE")

# initialize the list which contains the new priorities which are original priority divided by
# time it takes to complete this task

time_used= 0

# Create new priority list containing ratios

priority = []

# add for loop to create this list
# use method .append() to  create elements of the new priority list
for i in range(n_tasks):
    priority.append(old_priority[i]/time[i])

for i in range (n_tasks):
    max_value = max(priority)
    max_index = priority.index(max_value)
    task_time = time[max_index]
    priority[max_index]= 0
    if time_used + task_time <= max_time:
        time_used += task_time
        print(f"{max_index+1}              {old_priority[max_index]}                {time_used}")
        if time_used == max_time:
            break
print(f"total time used = {time_used}")
```

```
TASK          PRIORITY        TIME TO COMPLETE
6              8                60
4              7                130
5              10                250
1              2                280
3              1                300
7              5                450
total time used = 450
```

```
# The two algorithms produced different results because they used
# different criteria to complete the tasks. The first algorithm used
# priority to determine which tasks to complete (with constraints on time)
# while the second algortihm used a ratio of priority to time to determine
# which tasks to complete.

# the second algorith allowed more tasks to be comleted in 8 hours.
```